HTML & CSS

A beginner's cheat sheet: the tags you need, and the CSS you must understand

What's inside

Part 1 - HTML: the most-used tags with a one-line explanation each.
Part 2 - CSS (the main part): selectors, units, the box model, padding vs margin, borders, typography, display, position, flexbox, grid, and more, each with a short note on how to think about it.

> **Note:** HTML is the **structure** (what is on the page). CSS is the **style** (how it looks and where it sits). Think of HTML as the skeleton and CSS as the clothes and layout.

Part 1: HTML Basics

Every page is built from **tags**. Most come in pairs: an opening tag, content, and a closing tag, e.g. `<p>Hello</p>`. Some, like `<img>` and `<br>`, have no closing tag.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Page</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Hello, world</h1>
    <p>My first page.</p>
  </body>
</html>
```

Tag	What it does
`<!DOCTYPE html>`	Tells the browser this is a modern HTML5 document.
`<html>`	Root element. Everything else lives inside it.
`<head>`	Hidden info about the page: title, CSS links, meta tags.
`<title>`	Text shown on the browser tab.
`<meta>`	Metadata, e.g. character set and the mobile viewport.
`<link>`	Connects an external file, usually your CSS.
`<body>`	Everything the user actually sees.
`<h1> to <h6>`	Headings. h1 is the biggest and most important; use one h1 per page.

Tag	What it does
`<p>`	A paragraph of text.
`<a href="...">`	A link. The href attribute is the destination.
`<img src="..." alt="...">`	An image. alt describes it for screen readers and broken images.
`<div>`	Generic **block** container. Takes the full width and starts a new line.
`<span>`	Generic **inline** container. Only as wide as its content.
`<ul> <ol> <li>`	Unordered (bullets) list, ordered (numbers) list, and list items.
`<strong> <em>`	Bold (important) and italic (emphasis) text.
`  <hr>`	Line break and horizontal line.
`<button>`	A clickable button.
`<form> <input> <label>`	Form wrapper, input field (type="text", "email", etc.), and the caption for an input.
`<table> <tr> <th> <td>`	Table, row, header cell, data cell.
`<header> <nav> <main> <section> <footer>`	**Semantic** containers. Work like div but describe their purpose (better for SEO and accessibility).

Note: Two attributes you will use constantly: `class` (reusable label for styling, many elements can share it) and `id` (a unique label, only one element per page).

Part 2: CSS

CSS is a list of rules. Each rule picks elements with a **selector** and sets **properties** inside curly braces.

```
selector {
  property: value;
  property: value;
}

h1 {
  color: navy;
  font-size: 32px;
}
```

1. Three ways to add CSS

External file is the one you should use almost always.

```
<!-- 1. External (best): in <head> -->
<link rel="stylesheet" href="style.css">

<!-- 2. Internal: in <head> -->
<style> p { color: red; } </style>

<!-- 3. Inline (avoid): on the element -->
<p style="color: red;">Hello</p>
```

Note: If two rules clash, the more specific one wins (inline > id > class > element). If they are equally specific, the one written later wins. This is the 'cascade' in Cascading Style Sheets.

2. Selectors: choosing what to style

Selector	Meaning
`p`	Element selector: every <p>.
`.card`	Class selector: every element with class="card".
`#header`	ID selector: the one element with id="header".
`div p`	Descendant: any p *inside* a div (at any depth).
`div > p`	Child: only p that is a *direct* child of div.
`h1, h2, h3`	Grouping: apply the same rule to several selectors.
`a:hover`	Pseudo-class: when the mouse is over a link. Also :focus, :first-child, :nth-child(2).
`*`	Universal: every element.

Note: Prefer classes for styling. Ids are too specific and hard to override; element selectors are too broad.

3. Colors and units

Colors can be written as names, hex, rgb, or hsl. Sizes use units; know which are fixed and which are relative.

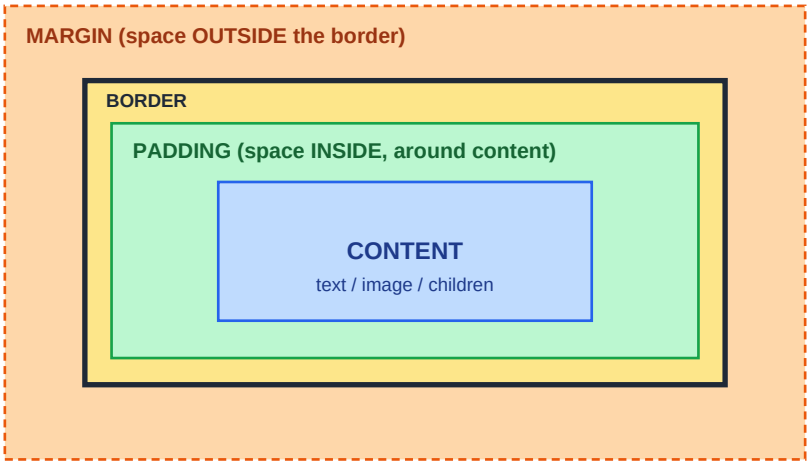
```
color: red;                   /* name */
color: #2563eb;               /* hex */
color: rgb(37, 99, 235);      /* rgb */
color: rgba(0, 0, 0, 0.5);    /* with transparency */

width: 300px;   /* fixed pixels */
width: 50%;     /* % of the PARENT's size */
font-size: 1.5rem;  /* rem: relative to the root (html) font size */
font-size: 2em;     /* em: relative to the parent's font size */
height: 100vh;      /* 100% of the viewport (screen) height */
width: 100vw;       /* 100% of the viewport width */
```

Note: Use **rem** for font sizes and spacing (scales nicely with user settings), **%** or **fr** for flexible layouts, and **px** for small fixed things like borders.

4. The Box Model (the most important idea in CSS)

Every element is a rectangular box made of four layers, from the inside out: **content**, **padding**, **border**, and **margin**.




```
.box {
  width: 200px;
  padding: 20px;
  border: 4px solid black;
  margin: 30px;
}
```

Note: By default **width** only sets the content width. So the box above is really 200 + 20*2 + 4*2 = 248px wide. That surprises everyone at first, and the fix is the next property.

5. box-sizing: border-box (always set this)

Makes width and height include padding and border, so a 200px box stays 200px. Put this at the top of every stylesheet.

```
* {
  box-sizing: border-box;
}
```

Note: With border-box, adding padding shrinks the content area instead of growing the box. Layouts become predictable.

6. Padding: space INSIDE the element

Padding is the gap between the content and the border. The element's background color **fills** the padding.

```
padding: 20px;                /* all 4 sides */
padding: 10px 30px;           /* top+bottom | left+right */
padding: 10px 20px 30px;      /* top | left+right | bottom */
padding: 10px 20px 30px 40px; /* top right bottom left (clockwise) */
padding-left: 15px;           /* single side */
```

Note: Shorthand order is clockwise from the top: **T**op, **R**ight, **B**ottom, **L**eft. Remember it as 'TRouBLe'. Same order for margin and border.

7. Margin: space OUTSIDE the element

Margin pushes *other* elements away. It is always transparent (no background color).

```
margin: 20px;
margin: 0 auto;      /* centers a block element horizontally (needs a width) */
margin-top: 40px;
margin: -10px;       /* negative margins are allowed; they pull things closer */
```

Note: Vertical margins between two stacked blocks **collapse**: a 30px bottom margin touching a 20px top margin gives a 30px gap, not 50px. The bigger one wins. This only happens vertically, and not inside flex or grid containers.

8. Padding vs Margin: the difference

	Padding	Margin
Where is the space?	**Inside** the border, around the content	**Outside** the border, between elements
Background color	Is painted behind it	Never painted (transparent)
Clickable area	Counts as part of the element (a bigger button)	Not part of the element
Can be negative?	No	Yes
Can be auto?	No	Yes (margin: 0 auto centers)
Collapses vertically?	No	Yes, between adjacent blocks
Use it to...	Give content breathing room (buttons, cards)	Separate one element from its neighbours

Note: Quick test: *should the gap be part of this element (inside)?* Use padding. *Should it separate this element from another?* Use margin.

9. Border and border-radius

A line around the element, sitting between padding and margin. The shorthand is width, style, color.

```
border: 2px solid #333;        /* width style color */
border-bottom: 1px dashed gray;
border-radius: 8px;            /* rounded corners */
border-radius: 50%;            /* turns a square into a circle */
outline: 2px solid blue;       /* like border but takes no space */
```

Note: Border styles: solid, dashed, dotted, double, none. A border with no style will not show.

10. Width, height and limits

Control the size of the content box. min/max versions make layouts flexible.

```
width: 100%;
max-width: 1100px;   /* never wider than this */
min-height: 100vh;   /* at least a full screen tall */
height: auto;        /* default: grows with content */
```

Note: Use max-width instead of width for containers: it shrinks on small screens, but stops growing on big ones.

11. Typography

The properties that control text.

```
font-family: "Inter", Arial, sans-serif;  /* first choice, then fallbacks */
font-size: 1rem;
font-weight: 700;        /* 400 normal, 700 bold */
line-height: 1.6;        /* spacing between lines (unitless) */
text-align: center;      /* left | right | center | justify */
text-decoration: none;   /* remove link underline */
text-transform: uppercase;
letter-spacing: 0.05em;
color: #1f2937;
```

Note: Always list fallback fonts, ending with a generic family (sans-serif, serif, monospace). A line-height of 1.5 to 1.7 makes body text easy to read.

12. Backgrounds, shadows and opacity

Make elements visually distinct.

```
background-color: #eff6ff;
background-image: url("hero.jpg");
background-size: cover;          /* fill the box, crop if needed */
background-position: center;
background-repeat: no-repeat;
background: linear-gradient(to right, #2563eb, #7c3aed);
box-shadow: 0 4px 12px rgba(0,0,0,0.15);  /* x  y  blur  color */
opacity: 0.8;                    /* 0 invisible to 1 solid */
```

Note: box-shadow takes: horizontal offset, vertical offset, blur, color. Keep shadows soft and subtle.

13. display: how an element behaves in the flow

Decides whether an element takes a full line, sits inline with text, or disappears.

```
display: block;         /* full width, starts a new line (div, p, h1) */
display: inline;        /* only as wide as content; width/height ignored (span, a) */
display: inline-block;  /* sits inline, but accepts width/height/padding */
display: none;          /* removed completely from the page */
display: flex;          /* turns on flexbox (next section) */
display: grid;          /* turns on grid */
```

Note: 'display: none' removes the element and its space. 'visibility: hidden' hides it but keeps the empty space.

14. position and z-index

Lets you move an element out of its normal place.

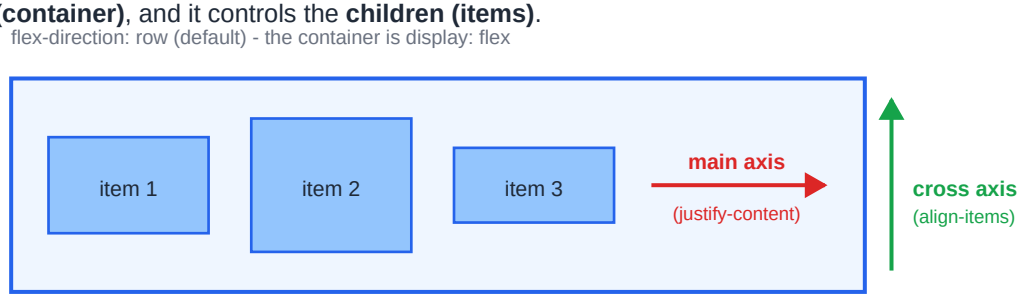
```
position: static;     /* default, normal flow */
position: relative;   /* normal flow, but can be nudged with top/left; also becomes
                         the anchor for absolute children */
position: absolute;   /* removed from flow; placed relative to nearest positioned parent */
position: fixed;      /* stuck to the screen even when scrolling (sticky navbars) */
position: sticky;     /* scrolls normally, then sticks at an offset, e.g. top: 0 */

.badge { position: absolute; top: 8px; right: 8px; z-index: 10; }
```

Note: The classic pattern: parent gets **position: relative**, child gets **position: absolute**, so the child is positioned inside the parent. z-index decides which overlapping element is on top (higher = in front) and only works on positioned elements.

15. Flexbox: one-dimensional layout

Flexbox lays out children in a **row or column** and gives you easy control over spacing and alignment. You turn it on for the **parent (container)**, and it controls the **children (items)**.




```
.container {
  display: flex;
  justify-content: space-between;  /* along the MAIN axis */
  align-items: center;             /* along the CROSS axis */
  gap: 16px;                       /* space between items */
}
```

Container properties (put on the parent)

Property	What it does
`flex-direction`	row (default) | column | row-reverse | column-reverse. Sets which way the main axis points.
`justify-content`	Spacing along the **main** axis: flex-start | center | flex-end | space-between | space-around | space-evenly.
`align-items`	Alignment along the **cross** axis: stretch (default) | center | flex-start | flex-end.
`flex-wrap`	nowrap (default) | wrap. Lets items drop to the next line when there is no room.
`gap`	Space between items. Cleaner than adding margins.

Item properties (put on the children)

Property	What it does
`flex: 1`	Let this item grow to share the leftover space. Two items with flex: 1 are equal width.
`flex-grow`	How much of the free space this item takes, relative to its siblings.
`flex-shrink`	Whether the item may shrink when space is tight (default 1).
`flex-basis`	The starting size before growing/shrinking.
`align-self`	Override align-items for just this one item.
`order`	Change the visual order without touching the HTML.

```
/* Perfectly centered content (the famous one) */
.center {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

/* Navbar: logo left, links right */
nav { display: flex; justify-content: space-between; align-items: center; }
```

Note: justify = main axis, **align** = cross axis. If you switch to flex-direction: column, the axes swap, so justify-content now works vertically. Most flexbox confusion comes from forgetting this.

16. CSS Grid: two-dimensional layout

Grid lays things out in rows **and** columns at the same time. Use flexbox for one direction (a row of buttons), grid for whole-page or card-grid layouts.

```
.cards {
  display: grid;
  grid-template-columns: repeat(3, 1fr);   /* 3 equal columns */
  gap: 20px;
}

/* Responsive without media queries */
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 20px;
}
```

Note: 1fr means 'one share of the free space'. repeat(3, 1fr) = three equal columns. An item can span columns with grid-column: span 2.

17. overflow and other handy properties

What happens when content is bigger than its box, plus a few small things you will use daily.

```
overflow: hidden;      /* clip extra content */
overflow: auto;        /* add scrollbars only when needed */
cursor: pointer;       /* hand cursor on hover (buttons, cards) */
list-style: none;      /* remove bullets */
object-fit: cover;     /* images fill their box without stretching */
max-width: 100%;       /* make images responsive: img { max-width: 100%; } */
```

18. Transitions and hover effects

Animate a change smoothly instead of snapping.

```
.btn {
  background: #2563eb;
  transition: background 0.3s ease, transform 0.2s ease;
}
.btn:hover {
  background: #1d4ed8;
  transform: translateY(-2px);   /* lift up slightly */
}
```

Note: Put the transition on the *normal* state (.btn), not only on :hover, so it animates both in and out.

19. Media queries: responsive design

Apply styles only when the screen matches a condition. Design for mobile first, then add rules for larger screens.

```
.cards { display: grid; grid-template-columns: 1fr; }   /* mobile: 1 column */

@media (min-width: 768px) {
  .cards { grid-template-columns: repeat(2, 1fr); }     /* tablet: 2 */
}
@media (min-width: 1024px) {
  .cards { grid-template-columns: repeat(3, 1fr); }     /* desktop: 3 */
}
```

Note: Do not forget the viewport meta tag in your HTML head, otherwise phones will ignore your media queries.

Starter stylesheet to paste into every project

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
body {
  font-family: system-ui, Arial, sans-serif;
  line-height: 1.6;
  color: #1f2937;
}
img { max-width: 100%; display: block; }
a { color: inherit; text-decoration: none; }
```

Common beginner mistakes

• Forgetting **box-sizing: border-box**, then wondering why boxes overflow.
• Using margin where padding was needed (and the background color does not fill the gap).
• Putting **justify-content** on the flex *item* instead of the *container*.
• Using **display: flex** on the parent but then styling the wrong element.
• Centering with **margin: auto** on an element that has no width.
• Overusing ids and !important, which makes your CSS hard to change later.
• Not opening the browser DevTools (right click, Inspect). It shows the box model of any element live.